



Nomes: Cainan Bastos Da Silva **Nº06**
Maria Eduarda Felix Inocência **Nº21**

Turma: 3 ano A

Arquitetura em Camadas parte do DEV

1. Visão Geral e Contexto

1.1 Introdução: Qual é o objetivo do sistema e qual problema ele resolve?

O Sistema de Gestão de Eventos (EventHub Solutions) tem como objetivo centralizar e automatizar os processos relacionados ao gerenciamento de eventos acadêmicos e corporativos.

Atualmente, o cenário apresentado pela empresa fictícia demonstra diversos problemas operacionais, incluindo cadastro manual de participantes, ausência de autenticação, falhas no armazenamento de informações e dificuldades de integração entre sistemas.

O projeto visa solucionar essas limitações por meio de uma aplicação Back-End estruturada, escalável e preparada para futuras integrações.

O Time 2 é responsável pela implementação da arquitetura em camadas, estabelecendo uma base sólida para a manutenção e evolução do sistema.

1.2 Público-alvo: Quem vai usar a aplicação (administradores, clientes, fornecedores)?

O sistema foi projetado para atender diferentes perfis de usuários:

Administradores

Responsáveis pelo gerenciamento completo dos eventos, participantes e configurações do sistema.

Organizadores

Usuários responsáveis pela criação e administração de eventos específicos.

Participantes

Usuários que realizam inscrições e acompanham informações relacionadas aos eventos.

Equipes Técnicas

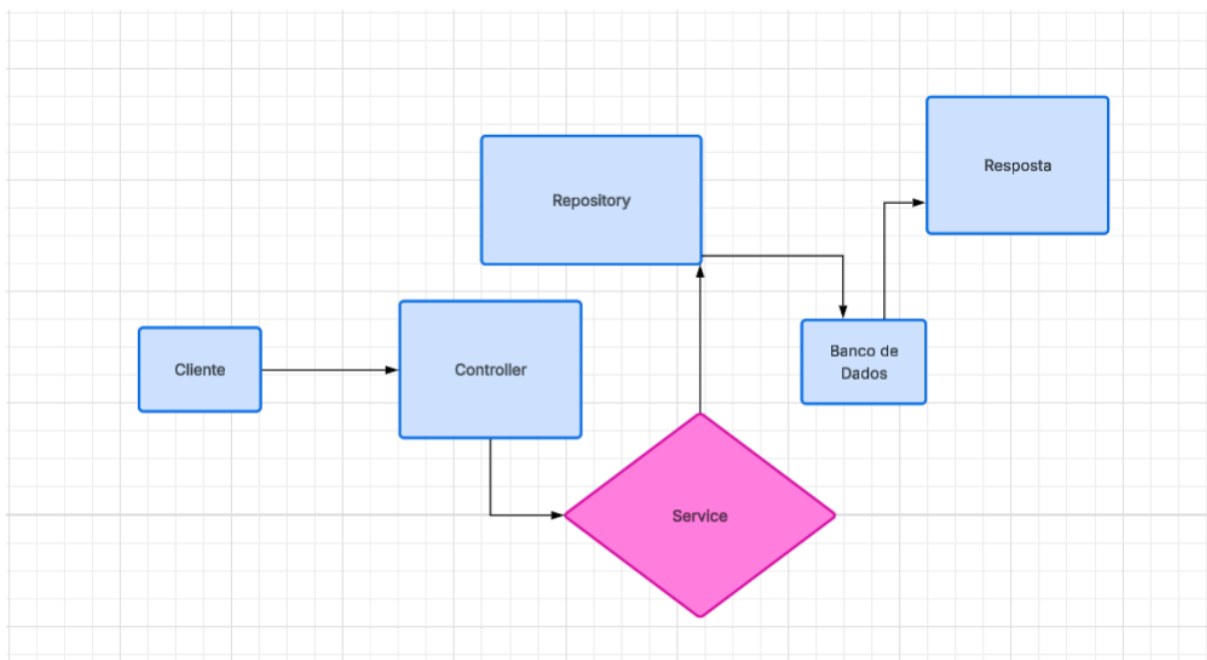
Desenvolvedores e administradores responsáveis pela manutenção e monitoramento da aplicação.

1.3 Regras de Negócio: Para garantir padronização e qualidade do sistema, foram definidas as seguintes regras de negócio:

Nº DA REGRA	REGRA	OBJETIVO
RN-001	Separação de Responsabilidades	Garantir organização, manutenção facilitada e reutilização de código.
RN-002	Responsabilidade da Camada Controller	Receber requisições HTTP; Encaminhar dados para a camada Service; Retornar respostas ao cliente; Definir os códigos de status HTTP.
RN-003	Responsabilidade da Camada Service	Centralizar as regras de negócio do sistema; Realizar validações necessárias; Processar informações recebidas do Controller; Encaminhar operações para o Repository.
RN-004	Responsabilidade da Camada Repository	A camada Repository deverá ser responsável exclusivamente pelo acesso aos dados.
RN-005	Padronização do Código	Os arquivos deverão seguir um padrão de nomenclatura e organização definido pela equipe para facilitar manutenção e integração com outros módulos.
RN-006	Integração entre Equipes	A arquitetura implementada deverá permitir integração com todos os módulos do

		sistema.
RN-007	Facilidade de Manutenção	A arquitetura deverá permitir alterações futuras sem necessidade de modificar múltiplas camadas simultaneamente.
RN-008	Refatoração Obrigatória	A equipe deverá demonstrar a evolução do sistema por meio da comparação
RN-009	Testabilidade	Cada camada deverá poder ser testada individualmente, facilitando a identificação de erros e a validação das funcionalidades do sistema.
RN-010	Escalabilidade	A arquitetura deverá permitir a adição de novas funcionalidades e módulos sem comprometer a estrutura existente do projeto.

2. Arquitetura do Sistema



2.1 Stack Tecnológica:

Back-End

- Node.js
- Express.js
- JavaScript

Ferramentas de Desenvolvimento

- Visual Studio Code
- Git
- GitHub
- Postman

Banco de Dados

Durante o desenvolvimento inicial:

- Estrutura simulada em memória (Array)

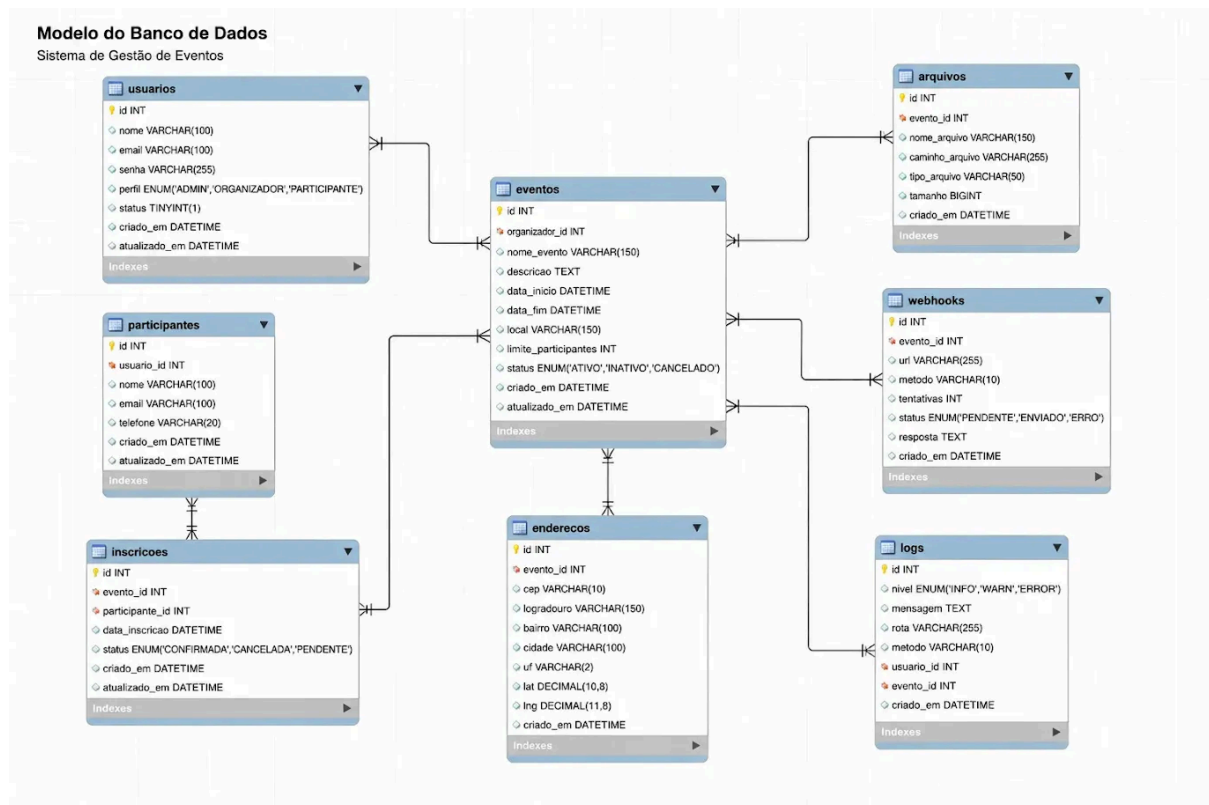
Para integração final:

- MySQL

Controle de Versão

- Git
- GitHub

3.1 Modelagem de Dados: Diagrama de Entidade-Relacionamento (DER).



4. Integrações e APIs

APIs de Terceiros: Quaisquer serviços externos que o sistema vai consumir (ex: gateways de pagamento como Stripe/Pagar.me, serviços de e-mail).

VIACEP: A API de terceiros utilizada será a do VIACEP, permitindo com que o usuário tenha o endereço preenchido automaticamente logo após colocar o número do CEP.

- **Documentação de API: Padrão de comunicação (REST, GraphQL, gRPC), formato de dados (JSON) e autenticação (JWT, OAuth).**

O padrão de comunicação da aplicação será através de API's REST, o formato de dados em JSON e a autenticação através de OAuth. Garantindo um envio padronizado, eficiente e seguro dos dados.